

Listas

En Python, los arreglos se conocen como listas, que son una estructura de datos que permite almacenar una colección de elementos ordenados. Las listas pueden contener elementos de diferentes tipos de datos, como enteros, flotantes, cadenas, booleanos, entre otros.

Las listas en Python son muy versátiles y flexibles, y se pueden modificar en tiempo de ejecución. Es decir, se pueden agregar, eliminar y modificar elementos en la lista. Además, las listas en Python son dinámicas, lo que significa que no se requiere definir previamente el tamaño de la lista.

Por ejemplo, se puede crear una lista de números enteros de la siguiente manera:

```
numeros = [1, 2, 3, 4, 5]
```

También se puede crear una lista que contenga diferentes tipos de datos:

```
lista = ['hola', 5, True, 3.14]
```

Es importante tener en cuenta que en Python, las listas se indexan a partir de cero, es decir, el primer elemento de la lista tiene un índice de 0, el segundo elemento tiene un índice de 1 y así sucesivamente.

Las listas en Python son muy útiles y se utilizan en una amplia variedad de aplicaciones y proyectos.

Tuplas

En Python, una tupla es otra estructura de datos que permite almacenar una colección de elementos ordenados, similar a una lista

Las tuplas se crean utilizando paréntesis () en lugar de corchetes [] como en las listas. Por ejemplo, se puede crear una tupla de números enteros de la siguiente manera:

```
tupla_numeros = (1, 2, 3, 4, 5)
```

También se puede crear una tupla que contenga diferentes tipos de datos:

```
tupla_datos = ('hola', 5, True, 3.14)
```

Al igual que con las listas, se pueden acceder a los elementos de una tupla mediante índices. Sin embargo, como las tuplas son inmutables, no se pueden agregar o eliminar elementos una vez creadas. Tampoco se pueden cambiar los valores de los elementos individuales.

Las tuplas se utilizan comúnmente para representar datos que no deberían cambiar, como coordenadas geográficas, valores constantes en una aplicación, fechas, entre otros.

Diferencias entre Listas y tuplas

Sintaxis: las listas se crean utilizando corchetes [] mientras que las tuplas se crean utilizando paréntesis ().

Listas

```
lista = [1, 2, 3, 4]
```

Tuplas

```
tupla = (1, 2, 3, 4)
```

Mutabilidad: las listas son mutables, lo que significa que se pueden agregar, eliminar y modificar elementos individuales en cualquier momento después de su creación. En cambio, las tuplas son inmutables, lo que significa que una vez creadas, no se pueden modificar los elementos individuales.

Listas

```
lista = [1, 2, 3, 4]  
lista[0] = 5  
print(lista) # [5, 2, 3, 4]
```

Tuplas

```
tupla = (1, 2, 3, 4)  
tupla[0] = 5 # esto generaría un error, ya que las  
tuplas son inmutables
```

Diferencias entre Listas y tuplas

Eficiencia: las tuplas son más eficientes en términos de espacio y tiempo de ejecución que las listas.

Listas

```
lista = [1, 2, 3, 4]
```

Tuplas

```
tupla = (1, 2, 3, 4)
```

Uso: las listas son más adecuadas para almacenar elementos que se necesitan modificar después de su creación, mientras que las tuplas son más adecuadas para almacenar elementos que no necesitan ser modificados..

Listas

```
# se almacenan los nombres de los estudiantes en una lista
```

```
estudiantes = ['Juan', 'Ana', 'Pedro', 'Luisa']
```

```
# se agrega un nuevo estudiante a la lista  
estudiantes.append('Marta')
```

```
# se elimina un estudiante de la lista  
estudiantes.remove('Ana')
```

```
# se modifica el nombre de un estudiante en la lista  
estudiantes[0] = 'Juan Carlos'
```

Tuplas

```
# se almacenan los días de la semana en una tupla  
dias_semana = ('Lunes', 'Martes', 'Miércoles', 'Jueves',  
'Viernes', 'Sábado', 'Domingo')
```

```
# se accede al tercer día de la semana  
tercer_dia = dias_semana[2]
```

```
# no se puede agregar un nuevo día a la tupla  
# tampoco se puede eliminar un día o modificar el  
nombre de un día
```

Diferencias entre Listas y tuplas

En resumen, las listas son más flexibles y se utilizan cuando se necesitan elementos que se puedan modificar después de su creación. Las tuplas son más eficientes en términos de espacio y tiempo de ejecución y se utilizan cuando se necesitan elementos que no necesitan ser modificados.

Funciones de arreglos en python

append(): Agrega un elemento al final de la lista.

```
frutas = ["manzana", "banana", "cereza"]  
frutas.append("naranja")  
print(frutas) # output: ["manzana", "banana", "cereza", "naranja"]
```

insert(): Agrega un elemento en una posición específica de la lista..

```
frutas = ["manzana", "banana", "cereza"]  
frutas.insert(1, "naranja")  
print(frutas) # output: ["manzana", "naranja", "banana", "cereza"]
```

Funciones de arreglos en python

remove(): Elimina el primer elemento de la lista que coincida con el valor dado.

```
frutas = ["manzana", "banana", "cereza"]  
frutas.remove("banana")  
print(frutas) # output: ["manzana", "cereza"]
```

pop(): Elimina el elemento en la posición dada y lo devuelve. Si no se especifica una posición, se elimina el último elemento de la lista.

```
frutas = ["manzana", "banana", "cereza"]  
frutas.pop(1)  
print(frutas) # output: ["manzana", "cereza"]
```

Funciones de arreglos en python

index(): Devuelve la posición del primer elemento que coincide con el valor dado.

```
frutas = ["manzana", "banana", "cereza"]  
posicion = frutas.index("banana")  
print(posicion) # output: 1
```

count(): Devuelve el número de veces que aparece el elemento dado en la lista.

```
frutas = ["manzana", "banana", "cereza", "banana"]  
conteo = frutas.count("banana")  
print(conteo) # output: 2
```


Funciones de arreglos en python

sort(): Ordena los elementos de la lista en orden ascendente.

```
numeros = [3, 1, 4, 1, 5, 9, 2, 6, 5]  
numeros.sort()  
print(numeros) # output: [1, 1, 2, 3, 4, 5, 5, 6, 9]
```

reverse(): Invierte el orden de los elementos de la lista.

```
frutas = ["manzana", "banana", "cereza"]  
frutas.reverse()  
print(frutas) # output: ["cereza", "banana", "manzana"]
```

Funciones de arreglos en python

copy(): Devuelve una copia de la lista.

```
frutas = ["manzana", "banana", "cereza"]  
copia = frutas.copy()  
print(copia) # output: ["manzana", "banana", "cereza"]
```

clear(): Elimina todos los elementos de la lista.

```
frutas = ["manzana", "banana", "cereza"]  
frutas.clear()  
print(frutas) # output: []
```

Estas son solo algunas de las funciones más comunes de las listas en Python. Existen muchas otras funciones disponibles, y cada una tiene sus propios parámetros y efectos secundarios.

